

Análisis y Diseño con Patrones

PEC 1: Principios de Diseño y Patrones de Análisis

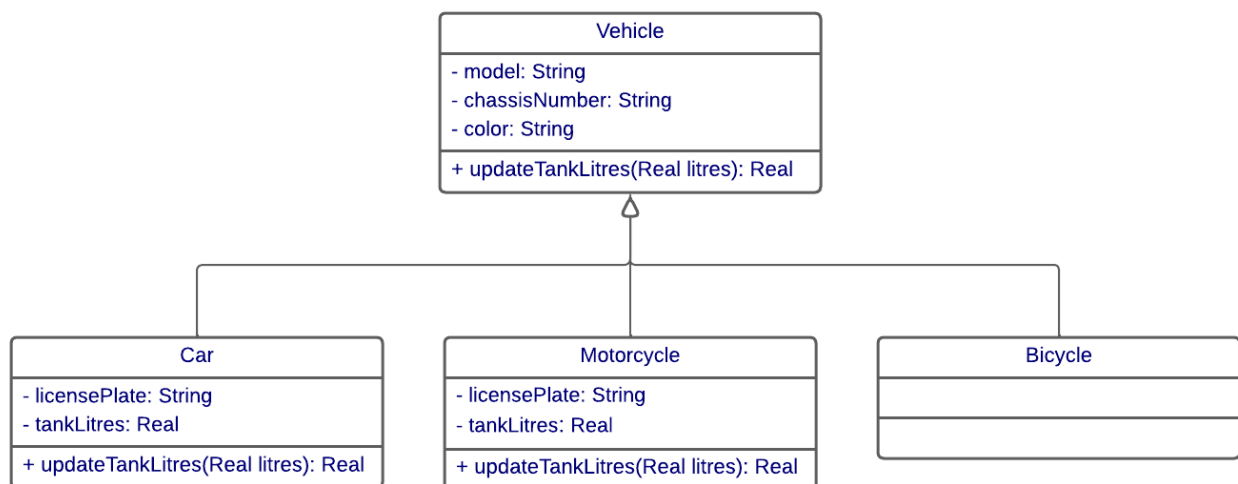
Pregunta 1 (30% puntuación)

Enunciado

Una empresa de vehículos nos propone analizar y diseñar el software que tienen. Se necesita registrar los diferentes tipos de vehículos que existen ya que cada uno implementará sus funciones de manera distinta. Por cada vehículo tenemos que registrar modelo, bastidor y color, además hay algún tipo de vehículo que tiene número de matrícula y el carburante que tiene en el depósito. Para modificar el carburante que tiene en el depósito hay implementada la siguiente función:

- *updateTankLitres(Real litres): Real* que sirve para añadir/restar carburante al vehículo y devuelve el número de litros que tiene el depósito.

Diagrama de clases



Restricciones de integridad

- Un vehículo se identifica por el número de bastidor (*chassisNumber*)
- La matrícula de los coches y de las motos (*licensePlate*) tiene que ser una matrícula española

Pseudocódigo

```
public class Vehicle {  
    private String model;  
    private String chassisNumber;  
    private String color;  
  
    public Real updateTankLitres(Real litres) {  
        return 0;  
    }  
}  
  
public class Car extends Vehicle {  
    private String licensePlate;  
    private Real tankLitres;  
  
    public Real updateTankLitres(Real litres) {  
        tankLitres= tankLitres + litres;  
        return tankLitres;  
    }  
}  
  
public class Motorcycle extends Vehicle {  
    private String licensePlate;  
    private Real tankLitres;  
  
    public Real updateTankLitres(Real litres) {  
        tankLitres= tankLitres+ litres;  
        return tankLitres;  
    }  
}  
  
public class Bicycle extends Vehicle {  
  
}
```

Antes de analizar y diseñar este software hay que repasar los principios de diseño, ya que son los que hay que cumplir para que un diseño sea de calidad.

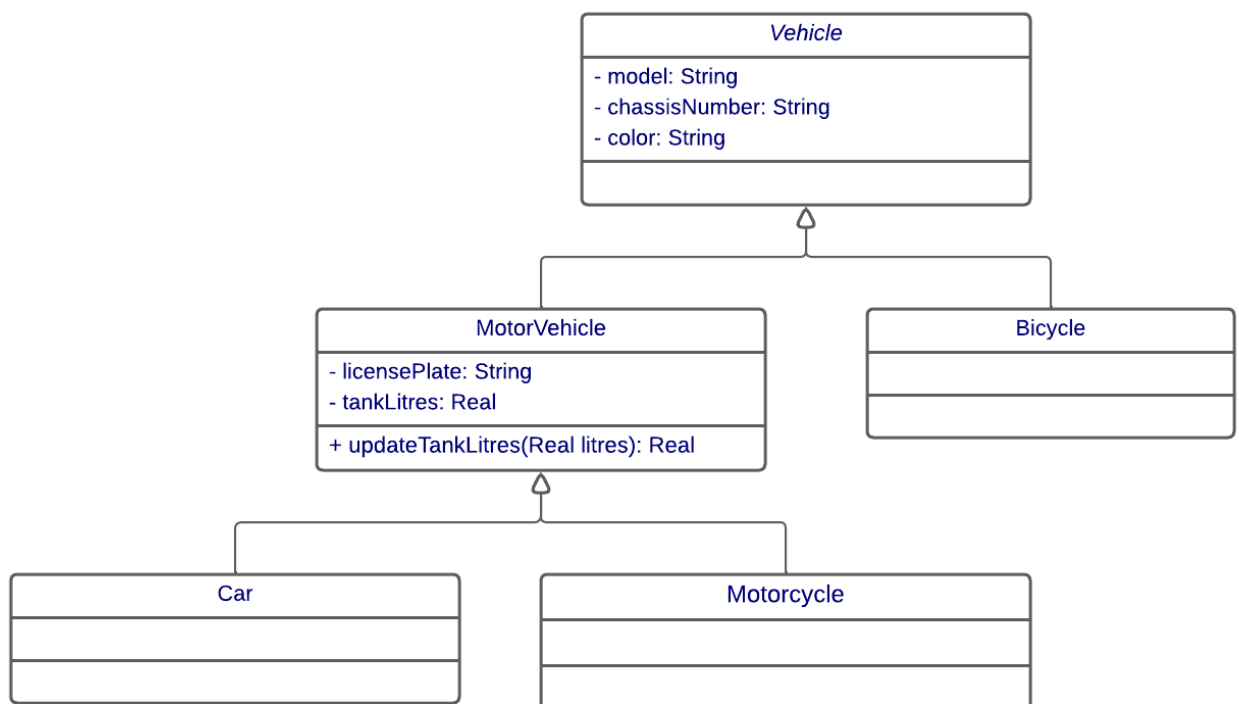
Se pide:

- a) Indica si este diseño satisface o no la Segregación de interfaces (ISP, *Interface-segregation principle*) y razona tu respuesta.

- b) Indica si este diseño satisface o no el principio de diseño No-repetición (DRY, *Don't Repeat Yourself*) y razona tu respuesta.
- c) Proporciona el análisis y diseño del software para que los principios de los anterior apartados se cumplan (sino se cumplían ya). El diseño debe incluir: modificación del diagrama de clases y del pseudocódigo de la operación *updateTankLitres(Real litres): Real*.

Solución

- a) El principio de Segregación de interfaces (ISP) no se satisface porque la clase Bicycle depende de la clase Vehicle y esta tiene funciones que no tendría porque ver Bicycle ya que no las va a usar.
- b) Este diseño no satisface el principio de No-repetición ya que en la clase Car y Motorcycle se repiten los atributos y la implementación de la función *updateTankLitres(Real litres): Real*.
- c) Para cumplir los principios, el diseño que se propone es el siguiente:



Pseudocódigo

```

public abstract class Vehicle {
    private String model;
    private String chassisNumber;
    private String color;

}
  
```

```
public class MotorVehicle extends Vehicle {
    private String licensePlate;
    private Real tankLitres;

    public Real updateTankLitres(Real litres) {
        tankLitres= tankLitres+ litres;
        return tankLitres;
    }
}

public class Car extends MotorVehicle {

}

public class Motorcycle extends MotorVehicle {

}

public class Bicycle extends Vehicle {

}
```

Con esta solución no tenemos funciones en la interfaz Vehicle que no necesiten todas las subclases que dependen de ella y por lo tanto, cumplimos el principio de Segregación de interfaces. Además, hemos creado una nueva clase que implemente una única vez la función updateTankLitres y tenga los atributos de los vehículos de motor, cumpliendo así el principio de No repetición y siendo más fácil de mantener el software.

Pregunta 2 (30% puntuación)

Enunciado

Los principios de diseño son muy importantes a la hora de diseñar un software, ya que son los que hay que cumplir para que un diseño sea de calidad. Uno de estos principios es el principio Abierto-Cerrado (*Open-closed principle*). Investiga en profundidad sobre este principio de diseño (puedes usar los módulos de la asignatura y las referencias que hay en el aula).

Una vez tengas claro de lo que trata este principio, se pide:

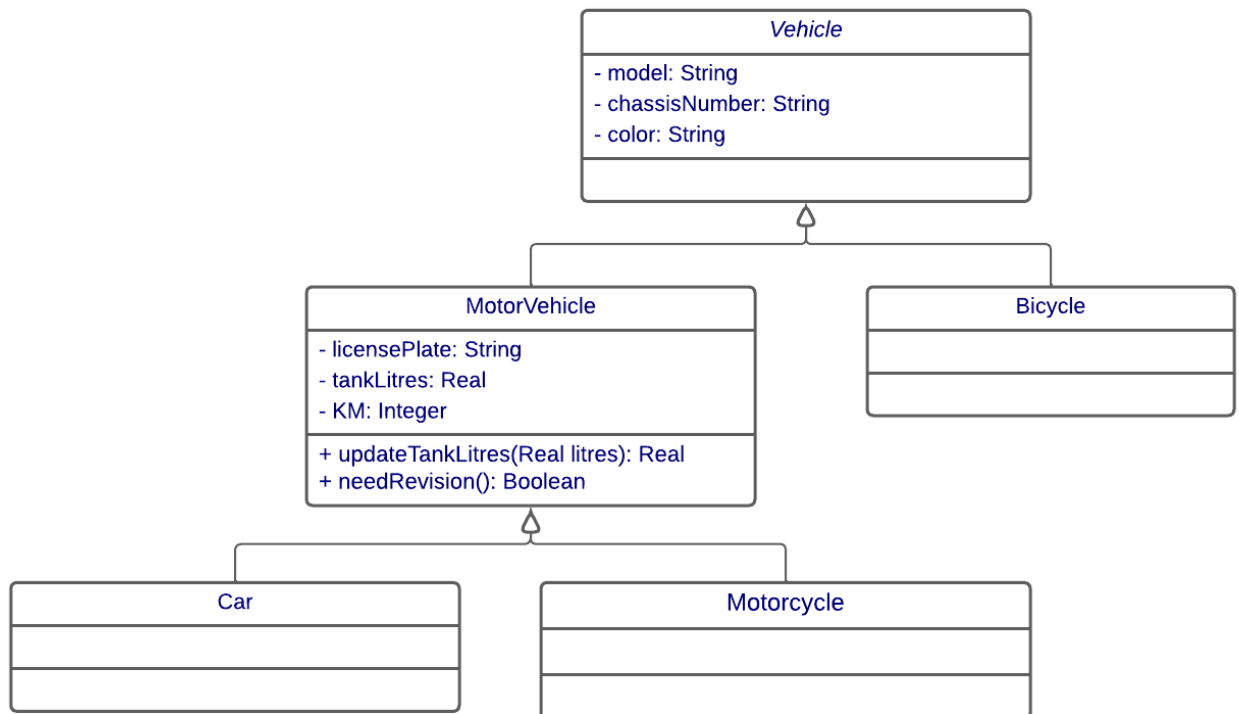
- a) Pon un ejemplo basado en el dominio del enunciado de la pregunta 1 que muestre cómo se viola este principio y explícalo. El ejemplo tiene que contener el diagrama de clases modificado, las restricciones de integridad nuevas (si tiene) y el diagrama de secuencia o pseudocódigo de una operación donde se muestre la violación, además de una explicación del ejemplo y de porqué se está violando el principio.

Nota: podéis modificar el diagrama de la pregunta 1 cómo creáis necesario (añadiendo clases nuevas, atributos, métodos, etc..) para que se viole el principio de Abierto-Cerrado.

- b) ¿Qué consecuencias tiene la violación de este principio?

Solución

- a) El principio Abierto-Cerrado nos dice que una entidad tiene que estar abierta a extensión pero cerrada a modificación. Esto significa que por ejemplo cuando hay una herencia las funciones en la superclase tienen que tener en cuenta que igual podría tener una subclase más en un futuro, por lo que cada función tendrá que implementarse en las subclases si es específica de cada tipo o en la superclase si es genérica. Ejemplo que no satisface el principio Abierto-Cerrado:



Pseudocódigo

```

public class MotorVehicle extends Vehicle {
    private String licensePlate;
    private Real tankLitres;
    private Integer KM;

    public Boolean needRevision() {
        if (typeof(self)==Car and KM>20000) {
            return True;
        }
        elseif (typeof(self)==Motorcycle and KM>10000) {
            return True;
        }
        else return False;
    }
}
    
```

En este ejemplo podemos ver que MotorVehicle no está cerrado a modificación ni abierto a extensión, ya que si añadimos otro tipo de vehículo de motor tendríamos que modificar la función dentro de la clase MotorVehicle. Para cumplir el principio Abierto-Cerrado tendríamos que haber usado la herencia que tiene MotorVehicle, hacer el método abstracto y que cada una de sus subclases implementará como se hace el cálculo.

- b) La consecuencia de no cumplir el principio Abierto-Cerrado es no cumplir otros principios como la Ley de Demeter, ya que estamos hablando con desconocidos y accediendo a clases que no tendríamos por que ver su comportamiento. También al no cumplir este principio hacemos que el software sea más difícil de mantener y que cualquier cambio haga que no funcione correctamente.

Pregunta 3 (20% puntuación)

Enunciado

El departamento de análisis y datos de una empresa de ropa quiere analizar qué informes de los que ha proporcionado a otros departamentos se usan y con qué frecuencia. Para poder tener esta información se necesita tener todos los informes registrados detallando: identificador del informe (número), la descripción del informe y ubicación de este informe (descriptivo). También los usuarios que tienen: nombre, apellidos, identificador de trabajador (número único), departamento y correo electrónico. Con estas dos entidades hay que ver cómo se pueden relacionar para obtener el registro del tiempo que pasa un usuario en un informe concreto.

Tened en cuenta que un usuario puede abrir un informe pasar un tiempo viéndolo, cerrarlo y luego volver a abrirlo.

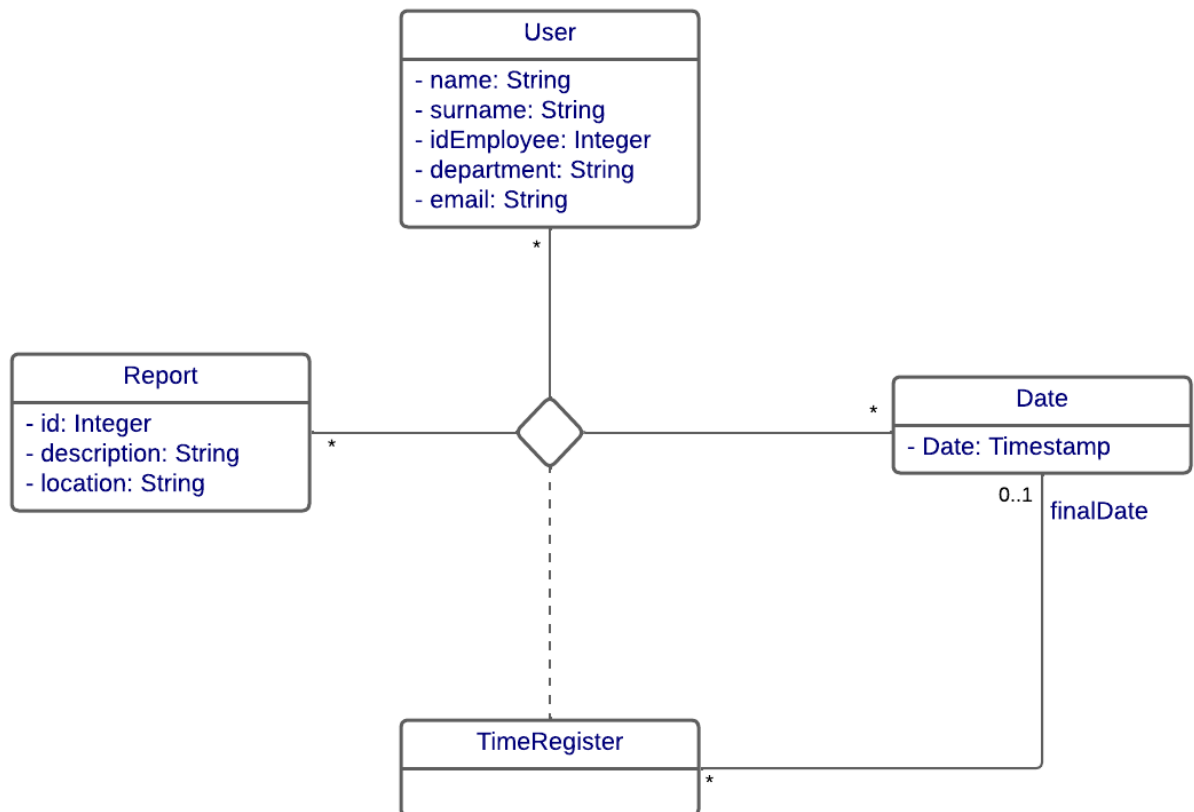
Se pide:

- Indicad qué patrones de análisis tenemos que tener en cuenta para implementar este requisito y justificad brevemente por qué.
- Proponed un diagrama estático de análisis donde se apliquen los patrones de análisis identificados en los requisitos descritos y las restricciones de integridad.

Nota: Si hace falta, podéis hacer alguna variación sobre los patrones tal y como están explicados en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

- a) El patrón de análisis más adecuado es el patrón asociación histórica donde tendremos la fecha (incluyendo día y hora) de inicio y finalización de un usuario y el informe que está usando. Este patrón nos permitirá registrar el histórico de los tiempos que pasa cada usuario en cada informe a lo largo del tiempo.
- b)



Restricciones de integridad

- La fecha de finalización de un TimeRegister debe ser superior a la fecha de inicio.
- El identificador del informe debe ser único.
- El identificador de empleado del usuario debe ser único.
- Las visualizaciones de un usuario a un informe determinado no deben solaparse.

Pregunta 4 (20% puntuación)

Enunciado

Los creadores de una aplicación de sudokus quieren ganar más dinero y por eso deciden poner anuncios si el usuario no paga la versión premium. Han contactado a una empresa de anuncios que les proporcionarán lo que necesitan, pero con el requisito de parametrizar según la edad del usuario. Es decir, cada anuncio estará destinado a ciertas edades y solo le saldrá al usuario que tenga la edad. Por lo tanto, necesitaremos registrar los datos mínimos del usuario: nombre, apellidos, correo electrónico (identificador único), premium (booleano) y edad; además de los anuncios se tendrán el identificador del anuncio (número), la descripción, la categoría (descriptivo) y las edades comprendidas entre edad mínima y edad máxima.

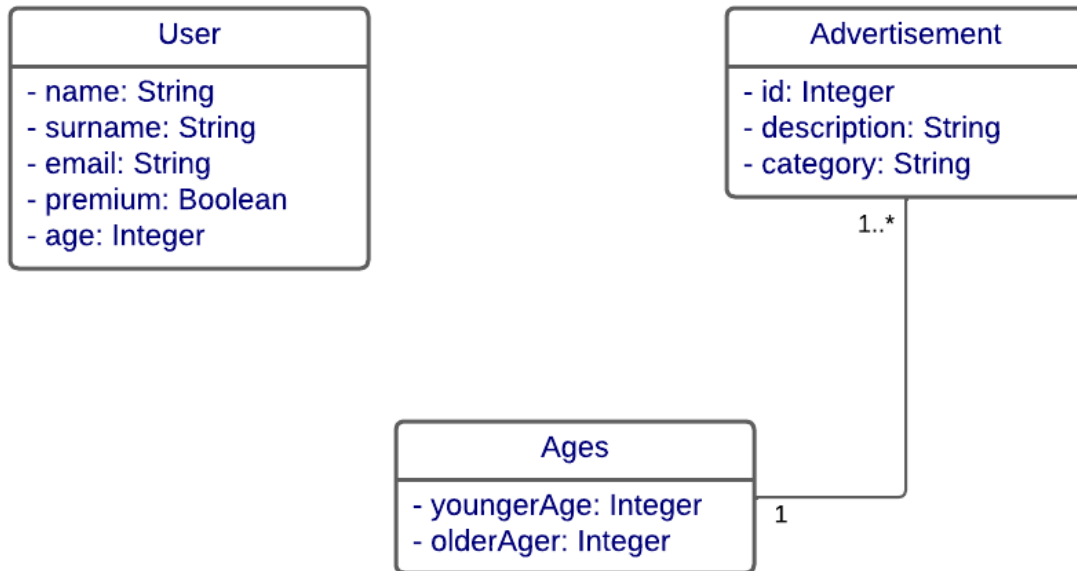
Se pide:

- a) Indicad qué patrones de análisis tenemos que tener en cuenta para implementar este requisito y justificad brevemente por qué.
- b) Proponed un diagrama estático de análisis donde se apliquen los patrones de análisis identificados en los requisitos descritos y las restricciones de integridad.

Nota: Si hace falta, podéis hacer alguna variación sobre los patrones tal y como están explicados en los materiales para poder satisfacer todos los requisitos enumerados.

Solución

- a) El patrón de análisis más adecuado es el patrón rango ya que tendremos el rango de edades que tiene un anuncio y el usuario lo relacionamos con su rango de edad. Así a través de la edad del usuario le saldrán solo los anuncios que toquen.
- b)



(Nota: si relacionamos User con Ages es correcto si añadimos la restricción de que la edad del usuario tiene que estar entre el rango de Ages)

Restricciones de integridad

- El identificador del anuncio debe ser único.
- El correo del usuario debe ser único.
- El youngerAge tiene que ser menor que olderAge.